PaySlate Project Flow — Screens, Data & Real-Time Updates

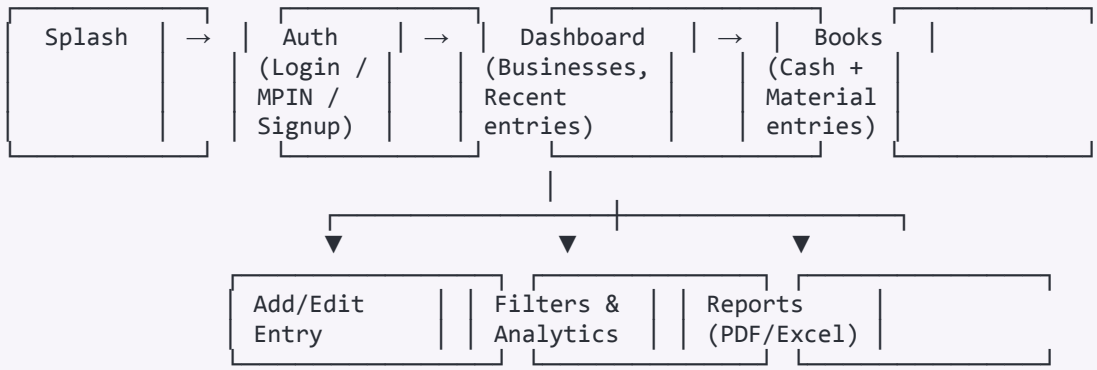
> A guided tour of how a user moves through PaySlate, and how data flows behind every screen they see.

This document is the bridge between **product behavior** and **clean architecture**. If the Architecture chapter tells you the *layers*, this doc tells you *how those layers cooperate* across real user journeys — from the splash screen to a downloaded PDF report.

> 🎯 **Use this when** you're designing a feature, writing a cubit, or trying to figure out which slice of the app a bug actually lives in.

High-level User Journey

The "happy path" through PaySlate looks like this:



Feature Map

Each feature module maps to a slice of user-facing functionality:

Feature module	What it owns
`auth`	Login, MPIN, signup, forgot password, OTP, biometric unlock
`splash` / `initial`	App boot, force-update check, route to login or dashboard

`dashboard`	Business + book list, recent entries card, analytics view
`book_navigation`	Per-book summary cards, category analytics, recent entries
`book_settings`	Members list, role management, share/invite
`cash_entry`	Cash book details, add/edit/delete cash entries, filters
`material_entry`	Material book details, add/edit/delete material entries
`account`	Customer/vendor account info, cash & material entries per account
`contacts`	Phone contact selection bottom sheets
`filters`	Shared filter sheet for entries (date, category, payment mode)
`notification`	FCM-driven in-app notifications, badge counts
`profile`	User profile, device security, app settings
`review`	Pending entry approvals (multi-user books)
`app_content`	Remote-config-style content (terms, privacy, FAQs)
`staging`	Staging-only debug panel (flavor-gated)
`error`	Global error / no-internet / force-update screens

Data Flow per Screen (Clean Architecture)

Let's trace **"Add Cash Entry"** end-to-end as the canonical example. Every other write flow follows the same shape:

```
┌─ Presentation ──────────────────────────────────────────────────────┐
│ AddCashEntryScreen                                                  │
│   ↓ on submit                                                       │
│ AddCashEntryCubit.add(entry)                                        │
│   ↓ injected via get_it                                             │
└─────────────────────────────────────────────────────────────────────┘
  │
┌─ Domain ────────────────────────────────────────────────────────────┐
│ AddCashEntry  use case        Returns Either<Failure, void>         │
│   ↓ calls                                                           │
│ CashEntryRepository (abstract)                                      │
└─────────────────────────────────────────────────────────────────────┘
  │
┌─ Data ──────────────────────────────────────────────────────────────┐
│ CashEntryRepositoryImpl                                             │
│   if (online) {                                                     │
│     remoteDataSource.addCashEntry(dto)                              │
│     localDataSource.cache(dto)                                      │
│   } else {                                                          │
│     localDataSource.queueForSync(dto)                               │
│   }                                                                 │
│   ↓                                                                 │
│   returns Right(unit) or Left(Failure)                              │
└─────────────────────────────────────────────────────────────────────┘
```

> 📌 **Takeaway:** the Presentation layer never speaks to Dio or Isar directly — it speaks to a Cubit, which speaks to a Use Case, which speaks to a Repository contract. The implementation is wired up in `app_bootstrap.dart`.

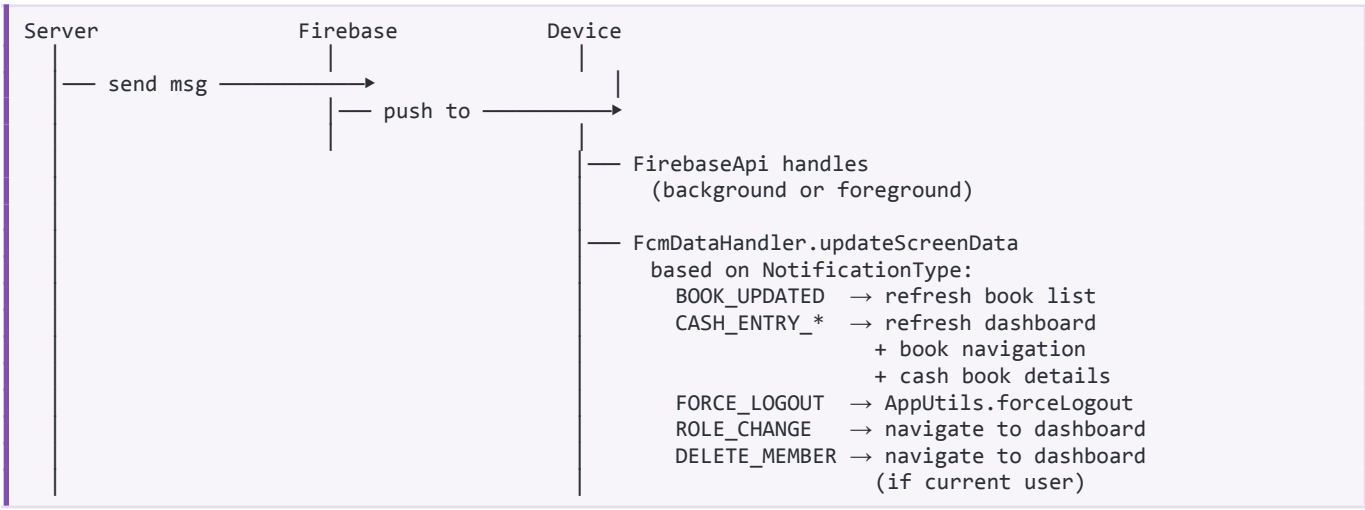
Offline-First Model

PaySlate is built so the user **never feels the network**. Here's how:

1. **Reads** — repositories return local data first, then fetch remote in the background and notify subscribers via cubit state updates.
2. **Writes** — when online, writes go through to remote + local cache simultaneously. When offline, writes go into a local sync queue.
3. **Sync manager** drains the queue when connectivity returns. UI shows a *"syncing X items"* banner during drain.
4. **List screens** must use `SmoothListView` + keyed rows + skeleton-only-when-cache-empty so the offline-first rebuild feels instant.

FCM (Push Notification) Flow

PaySlate uses Firebase Cloud Messaging to push real-time updates from the server. Here's the round-trip:



📂 Implementation lives in `lib/core/services/firebase/fcm_data_handler.dart`.

Navigation

PaySlate uses **`go_router`** with named routes. The key piece you should know:

> `RouteNameStorage` persists the latest route name so FCM handlers can decide whether to **refresh the visible screen** vs. **just queue data updates** in the background.

🔐 Auth-gated routes check token presence + validity via `AuthInterceptor`; `401` responses trigger logout and route to `/login`.

Multi-flavor Entry Points

PaySlate ships in **three flavors**, each with its own entry point:

Flavor	Entry point	Points to
dev	`lib/main_dev.dart`	Local API
staging	`lib/main_staging.dart`	Staging API + WebSocket
production	`lib/main_production.dart`	Live API

Each entry point sets its **flavor-specific API base URL, WebSocket URL, and feature flags**, then calls `app_bootstrap.dart`. See `pubspec.yaml` → `flavorizr` section for the per-flavor config.